



Lab 04 – Worksheet

Name: Ali Zain

ID: 11093

Section: T1

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes.

```
li    x10, 4          # Set n = 4
li    x2, 1
jal   x1, fact        # Call the factorial function
ecall
fact:
    li x11, 1          # initialize acc to 1
    bge x10, x0, L2    # if n >= 0, go to L2
L2:
    mul x11, x11, x10   # acc = acc * n
    addi x10, x10, -1   # n = n - 1
    bge x10, x2, L2     # if n >= 0, repeat loop

    mv x10, x11         # move result to x10
    jalr x0, 0(x1)      # return
```

**Add snip of relevant Registers / Memory locations where you think results are obtained. Make sure the irrelevant area of snip is cropped. Make sure the irrelevant area of snip is cropped.*



```
> CSR
  ✓ Integer
    x00 (zero) = 0
    x01 (ra)   = 12
    x02 (sp)   = 1
    x03 (gp)   = 268435456
    x04 (tp)   = 0
    x05 (t0)   = 0
    x06 (t1)   = 0
    x07 (t2)   = 0
    x08 (s0)   = 0
    x09 (s1)   = 0
    x10 (a0)   = 24
    x11 (a1)   = 24
    x12 (a2)   = 0
    x13 (a3)   = 0
```

Result of factorial of 4 being stored in x10

Comments

*A stack frame is not required here because the iterative fact function is a **leaf function** (it does not contain any jal instructions to call other functions). Since it never leaves its own execution scope until it finishes, it does not need to back up its return address (ra), nor does it need to preserve intermediate variables across function calls. All calculations can be done purely using the CPU's temporary and argument registers. It is highly memory efficient as it requires no memory space for a stack to store previous results.*



Task 2

Provide appropriately commented codes

```
li    x10, 4          # Set n = 4
li    x3, 1
li    sp, 0x100
jal   x1, fact        # Call the factorial function
end:
    j    end          # Infinite loop to end program
fact:
    addi sp, sp, -8    #adjust stack for 2 items
    sw   x1, 4(sp)     #save return address
    sw   x10, 0(sp)    #save argument n

    addi x5, x10, -1    #x5 = n - 1
    bge  x5, x3, L1     #if (n - 1) >= 0, go to L1

    addi x10, x0, 1     #return 1
    addi sp, sp, 8      #pop stack
    jalr x0, 0(x1)      #return

L1:
    addi x10, x10, -1   #argument = n - 1
    jal  x1, fact       #recursive call

    addi x6, x10, 0     #save result of fact(n-1)
    lw   x10, 0(sp)     #restore original n
    lw   x1, 4(sp)      #restore return address
    addi sp, sp, 8      #pop stack

    add  x10, x10, x6    #n * fact(n-1)
    jalr x0, 0(x1)      #return
```



Add screenshot of your results

```
x07 (t2) = 0
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 10
x11 (a1) = 0
x12 (a2) = 0
x13 (a3) = 0
```

Nth triangular number for 4 being stored in x10 register.

Comments

It uses the stack pointer to keep track of the nth number and then in the end it pops from the stack and adds the number, while through out the code a recursive approach is being performed to calculate that n-1 number, we use the stack so don't lose any data when returning the sum of numbers.



Task 3

Provide appropriately commented codes

```
.data
my_array: .word 42, 7, 19      # The unsorted array
size:     .word 3              # The number of elements

.text
.globl main
main:
    #setup arguments and call the sorting function
    la  x10, my_array          #base address of array
    lw  x11, size               #length of array

    jal x1, bubble_sort        #call the sort function

    # --- Code to Print the Array ---
    la  x05, my_array          # Load base address of array into a
temporary register
    lw  x06, size               # Load array size
    li  x07, 0                  # Initialize loop counter (i = 0)

print_loop:
    bge x07, x06, exit          # If i >= size, we are done printing, go
to exit

    #calculate address of my_array[i]
    slli x08, x07, 2            #multiply i by 4 (shift left logical by
2)
    add  x08, x05, x08          #add offset to base address

    # Load and print the integer
    lw  x09, 0(x08)             #load my_array[i] into a0 (argument
register for ecall)
    li  x10, 1                  #ecall code 1: print integer
    ecall

    #space character for readability
    li  x11, 32                 #ASCII code for space (' ')
    li  x12, 11                 #ecall code 11: print character
    ecall

    addi x07, x07, 1            #Increment counter (i++)
    j    print_loop             #Repeat for next element

exit:
```



```
    li    x13, 10                #exit program safely
    ecall

bubble_sort:
    #move stack pointer down by 16 bytes
    addi  sp, sp, -16

    #store registers relative to the new stack pointer
    sw    x1, 12(sp)
    sw    x8, 8(sp)
    sw    x9, 4(sp)

    #length is 0, skip sorting
    beq   x11, x0, sort_end

    li    x8, 0                  # i = 0
outer_loop:
    bge   x8, x11, sort_end
    mv    x9, x8                 # j = i
inner_loop:
    bge   x9, x11, inner_end

    #get a[i]
    slli  x28, x8, 2
    add   x28, x10, x28
    lw    x30, 0(x28)

    #get a[j]
    slli  x29, x9, 2
    add   x29, x10, x29
    lw    x31, 0(x29)

    #descending order check: if a[i] >= a[j], skip swap
    bge   x30, x31, skip_swap

    #perform swap
    sw    x31, 0(x28)
    sw    x30, 0(x29)

skip_swap:
    addi  x9, x9, 1
    j     inner_loop

inner_end:
    addi  x8, x8, 1
```



```
j    outer_loop

sort_end:
    #restore registers from their exact stack memory addresses
    lw    x9, 4(sp)
    lw    x8, 8(sp)
    lw    x1, 12(sp)

    #restore stack pointer
    addi sp, sp, 16
    jalr x0, 0(x1)
```

Add screenshots of your results

LAB4_TASK3.s

```
1  .data
2  my_array: .word 42, 7, 19  # The unsorted array
3  size:     .word 3         # The number of elements
4
5  .text
6  .globl main
7  main:
8      # Manually set the stack pointer to 0x100
9      li    sp, 0x000100
10
11     #setup arguments and call the sorting function
12     la     x10, my_array    #base address of array
13     lw     x11, size        #length of array
14
15     jal    x1, bubble_sort  #call the sort function
16
17 exit:
18     li     a7, 10          #exit program safely
19     ecall
20
21 bubble_sort:
```

Memory window:

Address	+0	+1	+2	+3
0x10000018	0	0	0	0
0x10000014	0	0	0	0
0x10000010	0	0	0	0
0x1000000C	3	0	0	0
0x10000008	19	0	0	0
0x10000004	7	0	0	0
0x10000000	42	0	0	0
0x0FFFFFFC	0	0	0	0
0x0FFFFFF8	0	0	0	0
0x0FFFFFF4	0	0	0	0
0x0FFFFFF0	0	0	0	0
0x0FFFFFEC	0	0	0	0
0x0FFFFFE8	0	0	0	0

The stored integers before the swap has happened.

LAB4_TASK1.bs

```
56 outer_loop:
57
58 inner_loop:
59     bge x9, x11, inner_end
60
61     #get a[i]
62     slli x28, x8, 2
63     add x28, x10, x28
64     lw x30, 0(x28)
65
66     #get a[j]
67     slli x29, x9, 2
68     add x29, x10, x29
69     lw x31, 0(x29)
70
71     #descending order check: if a[i] >= a[j], skip swap
72     bge x30, x31, skip_swap
73
74     #perform swap
75     sw x31, 0(x28)
76     sw x30, 0(x29)
77
78 skip_swap:
79     addi x9, x9, 1
80     j inner_loop
81
82 inner_end:
83     addi x8, x8, 1
84     j outer_loop
85
86 sort_end:
87     #restore registers from their exact stack memory address
```

Memory window:

Address	+0	+1	+2	+3
0x10000018	0	0	0	0
0x10000014	0	0	0	0
0x10000010	0	0	0	0
0x1000000C	3	0	0	0
0x10000008	19	0	0	0
0x10000004	7	0	0	0
0x10000000	42	0	0	0
0x0FFFFFFC	0	0	0	0
0x0FFFFFF8	0	0	0	0
0x0FFFFFF4	0	0	0	0
0x0FFFFFF0	0	0	0	0
0x0FFFFFEC	0	0	0	0
0x0FFFFFE8	0	0	0	0

19 is swapped with the place of 7 since its greater than 7.

The screenshot shows the Lab04-Git-AliZain IDE with the assembly code for LAB4_TASK3.s. The code implements a selection sort algorithm. The Memory window is open, showing the current address, value, and its representation in decimal and hexadecimal. The code is as follows:

```

56 outer_loop:
57
58 inner_loop:
59     bge x9, x11, inner_end
60
61     #get a[i]
62     slli x28, x8, 2
63     add x28, x10, x28
64     lw x30, 0(x28)
65
66     #get a[j]
67     slli x29, x9, 2
68     add x29, x10, x29
69     lw x31, 0(x29)
70
71     #descending order check: if a[i] >= a[j], skip swap
72     bge x30, x31, skip_swap
73
74     #perform swap
75     sw x31, 0(x28)
76     sw x30, 0(x29)
77
78 skip_swap:
79     addi x9, x9, 1
80     j inner_loop
81
82 inner_end:
83     addi x8, x8, 1
84     j outer_loop

```

The Memory window shows the current address 0x10000018, which contains the value 0. The display format is set to Decimal, and the bytes per row is 4.



Task 4

Provide appropriately commented codes

```
.data
    n: .word 4          #the Fibonacci term to calculate

.text
.globl main
main:
    lw    x10, n         #load n into argument register x10
    jal   x1, fib

    #print the final calculated Fibonacci number
    li    x17, 1         #ecall code for printing something
    ecall

    #exit program
    li    x17, 10        #ecall code 10 for exit
    ecall

fib:
    addi x2, x2, -12      #make room for 3 words on the stack
    sw    x1, 8(x2)       #save return address
    sw    x8, 4(x2)       #save x8 used to hold fib(n-1)
    sw    x10, 0(x2)      #save current n

    #base Case Check
    li    x5, 1
    ble   x10, x5, fib_done #if n <= 1, branch to end

    #recursive Step fib(n-1)
    addi x10, x10, -1     #n = n - 1
    jal   x1, fib         #call fib(n-1)
    mv    x8, x10         #temporarily store the result of fib(n-1) in x8

    #recursive Step 2 fib(n-2)
    lw    x10, 0(x2)      #reload original n from the stack
    addi x10, x10, -2     #n = n - 2
    jal   x1, fib         #call fib(n-2)

    #combing results
    add   x10, x10, x8     #x10 = fib(n-2) + fib(n-1)

fib_done:
    #eestore state from stack
    lw    x1, 8(x2)       #restore return address
```



```
lw    x8, 4(x2)    #restore x8
addi  x2, x2, 12    #deallocate stack space

jalr  x0, 0(x1)
```

Add screenshots of your results

```
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 3
x11 (a1) = 0
x12 (a2) = 0
x13 (a3) = 0
x14 (a4) = 0
x15 (a5) = 0
```

Comments:

The final result answer of fib(4) 3

fib(4) splits into *fib(3) + fib(2)*

fib(3) splits into *fib(2) + fib(1)*

fib(2) splits into *fib(1) + fib(0)*

The base case condition (ble x10, x5, fib_done) triggers for $n \leq 1$, returning the value of n directly.

fib(2) resolves to $1 + 0 = 1$

fib(3) resolves to $1 + 1 = 2$

fib(4) resolves to $2 + 1 = 3$

Link to github repo: <https://github.com/alizain2310/Lab04-Git-AliZain.git>





Lab 04 – Nest Procedures and Sorting Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	AR 7 Report Submission & Github
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary



			Report is submitted after due date.	The report is submitted timely.	figures, plots and tables.
--	--	--	-------------------------------------	---------------------------------	----------------------------